Strategy Pattern: Complaint SLA Calculation

Problem

The `Complaint` class originally used a **fixed SLA (24 hours)**.
Adding new rules (e.g., priority or VIP handling) would require complex `if-else` statements, making the code hard to maintain.

Solution

We applied the **Strategy Pattern** by:

- Creating an interface `SlaCalculator` with `calculateDeadline()`
- Moving SLA logic into separate strategy classes
- Passing the strategy to `Complaint` during object creation

Strategies Implemented

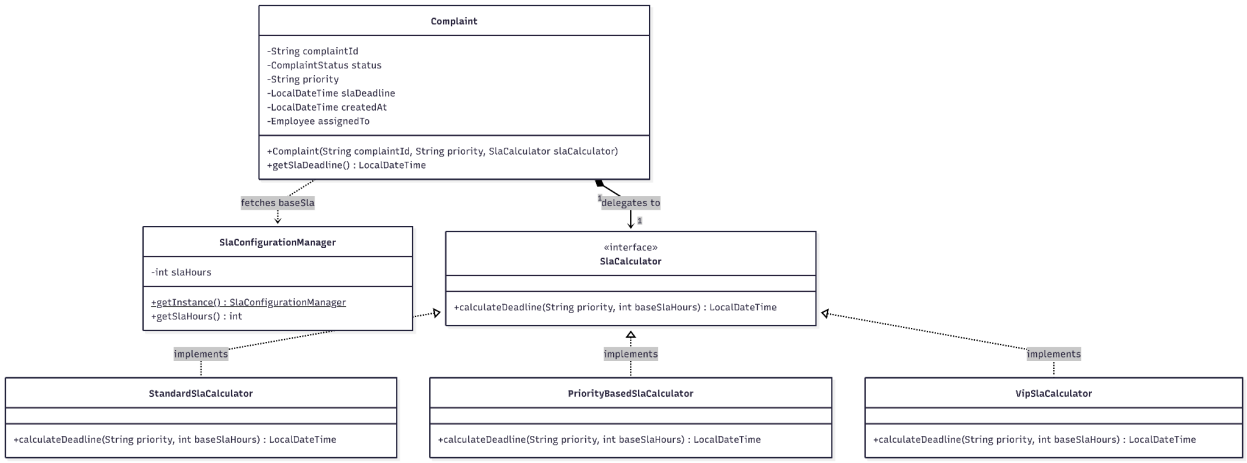
- **StandardSlaCalculator** → uses base SLA
- **PriorityBasedSlaCalculator** → adjusts SLA:
  - Critical → 4 hours
  - High → base / 2
  - Medium → base
  - Low → base × 2
- **VipSlaCalculator** → fixed 4-hour SLA

Changes in Complaint Class

- **Dependency Injection:** `Complaint` now receives an `SlaCalculator`
- **Delegation:** Deadline calculation is handled by the strategy
- Uses `SlaConfigurationManager` to get base SLA hours

Result

- **Encapsulation:** SLA logic is separated from `Complaint`
- **Flexibility:** Strategies can be changed at runtime
- **Scalability:** New rules can be added without modifying existing co



Strategy Pattern: Customer Segmentation Evaluation

Problem

The CustomerSegment class originally stored its criteria as a simple, unevaluable string.Adding programmatic segmentation logic directly into the class would require bloated if-else chains handling entirely distinct evaluation rules (e.g., total spending, physical address, or login frequency), which violates the Open/Closed Principle and makes the class hard to maintain.

Solution

We applied the **Strategy Pattern** by:

- Creating an interface SegmentationStrategy with an evaluate (Customer customer) method
- Moving the distinct evaluation logic into separate strategy classes
- Passing the chosen strategy to CustomerSegment during object creation (or via a setter)

Strategies Implemented

- **SpendingBasedStrategy** → evaluates if the customer's total order value meets a minimum threshold
- **ActivityBasedStrategy** → evaluates if the customer's last activity date falls within a maximum allowed number of days

- **GeographicStrategy** → evaluates if the customer's address/region matches a specific target region

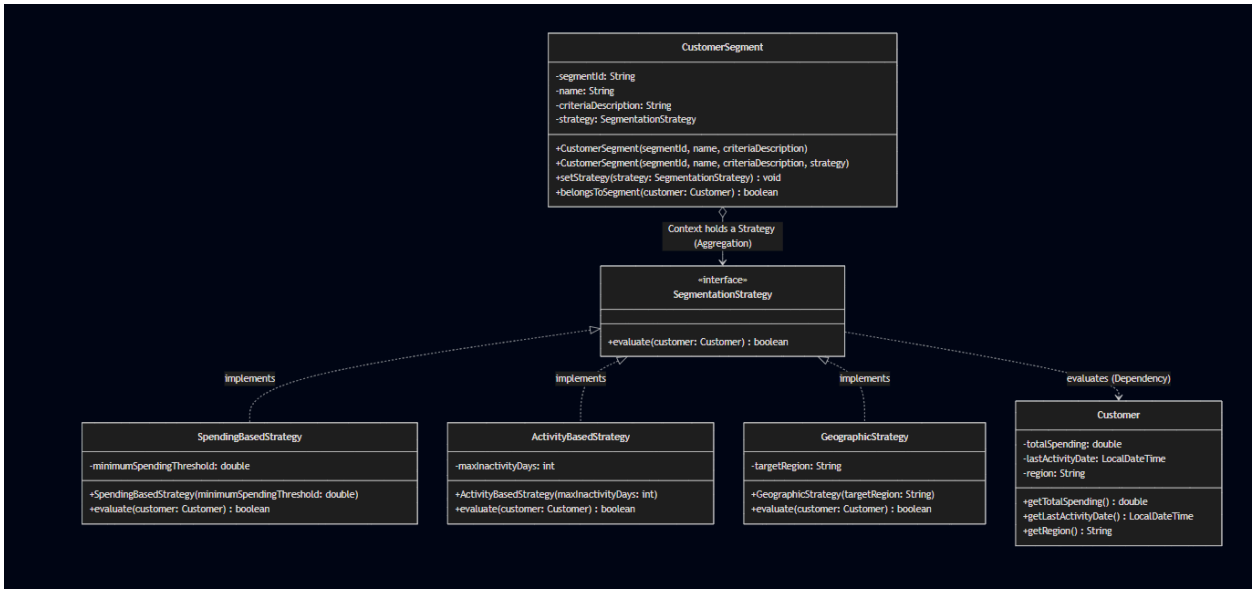
Changes in CustomerSegment Class

- **Dependency Injection**: CustomerSegment now receives a SegmentationStrategy variable
- **Delegation**: Programmatic membership checks (belongsToSegment()) are now handled by delegating the math entirely to the assigned strategy
- **Backwards Compatibility**: A secondary constructor was maintained to allow basic string-only segments to initialize without breaking

---

Result

- **Encapsulation**: Unique membership logic is safely separated from the CustomerSegment class
- **Flexibility**: Strategies can be swapped or assigned at runtime directly
- **Scalability**: Brand new segmentation rules (like VIP-based or Age-based) can be added smoothly without modifying any existing CustomerSegment code



Template Method Pattern: Report Generation Design

Problem

The `Report` class originally relied on a single `generate()` method that each subclass fully overrides.
This caused:

- No enforced structure between reports
- Duplicate logic across subclasses
- Inconsistent workflows (each report does things differently)
- Harder maintenance when adding new report types

Even though all reports follow the same idea:
collect data → **format data** → **render output**, this flow was not enforced.

Solution

We applied the **Template Method Pattern** by:

- Creating a **final method** `produceReport()` in the base `Report` class
- Defining the fixed workflow inside it:
  - `collectData()`
  - `formatData()`
  - `renderer.render(...)`
- Moving variable steps into **abstract methods (hooks)**
- Making `generate()` delegate to `produceReport()` to keep compatibility

Template Workflow (Locked in Parent)

```
produceReport():

    collectData()

    formatData()

    renderer.render(reportId, getReportType())
```

Hooks Implemented (Abstract Methods)

- `collectData()` → each report gathers its own data
- `formatData()` → each report formats its data differently
- `getReportType()` → identifies the report type

Reports Implemented

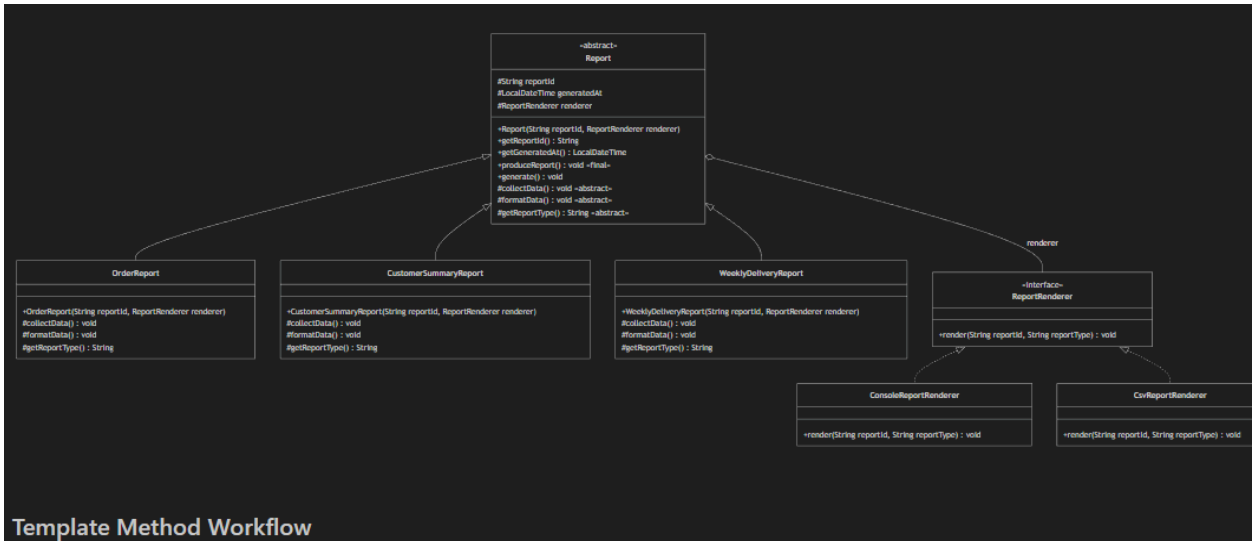
- **OrderReport**
  - Collects order records
  - Formats order-specific data
  - Returns `"OrderReport"`
- **CustomerSummaryReport**
  - Collects customer metrics
  - Formats summarized data
  - Returns `"CustomerSummaryReport"`
- **WeeklyDeliveryReport**
  - Collects weekly delivery data
  - Formats delivery statistics
  - Returns `"WeeklyDeliveryReport"`

Changes in Report Class

- Added `final produceReport()` → enforces workflow
- Added abstract methods: `collectData()`, `formatData()`, `getReportType()`
- Updated `generate()` → now calls `produceReport()`

Result

- **Consistency**: All reports follow the same execution flow
- **Encapsulation**: Common logic is centralized in the base class
- **Flexibility**: Subclasses only implement what varies
- **Maintainability**: No duplicated boilerplate
- **Safety**: `produceReport()` is final → cannot be altered by subclasses
- **Backward Compatibility**: Existing calls to `generate()` still work without changes



Template Method Pattern: Payment Processing Flow

Problem

The PaymentProcessor class originally did not enforce a consistent execution flow.
This caused:

- No enforced structure between different payment methods
- Duplicate logic across subclasses (like null/zero amount validation)
- Inconsistent workflows (e.g., some processors might capture funds without properly authorizing them first)
- Harder maintenance when adding new payment gateways Even though all payments follow the same fundamental idea: validate amount → authorize funds → capture funds, this critical flow was not strictly enforced.

Solution

We applied the **Template Method Pattern** by:

- Creating a final method executePayment(Money amount) in the base PaymentProcessor class
- Defining the fixed security and transaction workflow inside it:
  - validate(...)
  - authorize(...)
  - capture(...)
- Centralizing common validation logic within the base class
- Moving variable, payment-specific steps into abstract methods (hooks)

Template Workflow (Locked in Parent)

```
executePayment(amount):

  if (!validate(amount)) return false;

  if (!authorize(amount)) return false;

  capture(amount);

  return true;
```

Hooks Implemented (Abstract & Protected Methods)

- **`validate(amount)`** → `centralizes common checks (e.g., amount > 0) before proceeding`
- **`authorize(amount)`** → `abstract hook where each processor authorizes the payment differently`
- **`capture(amount)`** → `abstract hook where each processor finalizes or settles the transaction`

Processors Implemented

- **CardProcessor**
  - Inherits basic validation
  - Authorizes by contacting a simulated credit card network
  - Captures by settling previously authorized funds
- **CodProcessor (Cash on Delivery)**

- Inherits basic validation
- Authorizes automatically (pre-approved)
- Captures by marking the transaction for physical cash collection upon delivery

---

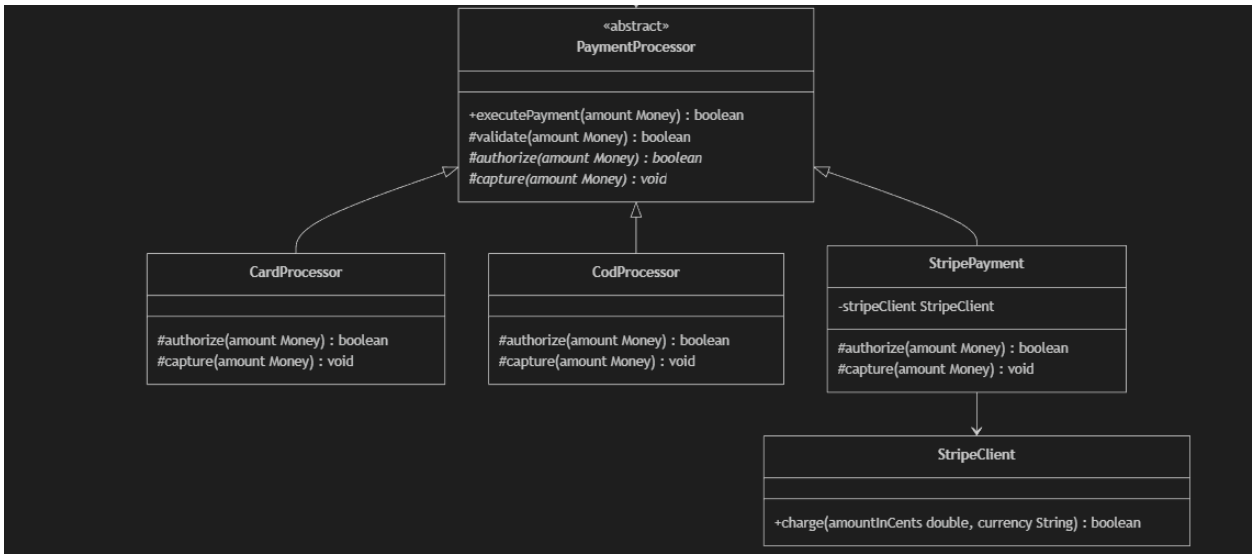
Changes in PaymentProcessor Class

- Added final executePayment(Money amount) → enforces workflow sequence and prevents bypasses
- Added common validate() method → strips boilerplate checks out of subclasses
- Added abstract methods: authorize() and capture() → strictly defines what subclasses must provide

---

Result

- **Consistency**: All payment transactions categorically follow the exact same execution flow
- **Encapsulation**: Common validation rules are centralized in the base class and managed in one place
- **Flexibility**: Subclasses only implement what varies (the specific network connection details)
- **Maintainability**: No duplicated boilerplate code across payment gateways
- **Safety**: executePayment() is final → critical transaction steps cannot be altered, reordered, or skipped by subclasses



Mediator Pattern: Order Fulfillment

Problem

Currently, coordinating a customer checkout requires manually managing Orders, Payments, Inventory stock, and Communication receipts. Managing this either directly within Main.java or by wiring the OrderController, PaymentController, ProductController, and CommunicationController to call each other creates a messy web of cross-dependencies. This scattered logic makes the codebase tightly coupled, brittle, and difficult to maintain or scale if the checkout process changes.

Solution

We applied the **Mediator Pattern** by:

- Creating a central OrderFulfillmentMediator class acting as an orchestrator with a processCheckout() method.
- Moving the complex, multi-domain workflow orchestration into this highly focused, centralized mediator hub instead of scattering it.
- Passing the raw domain controllers into the mediator upon initialization, allowing it to act on them externally.

Mediators Implemented

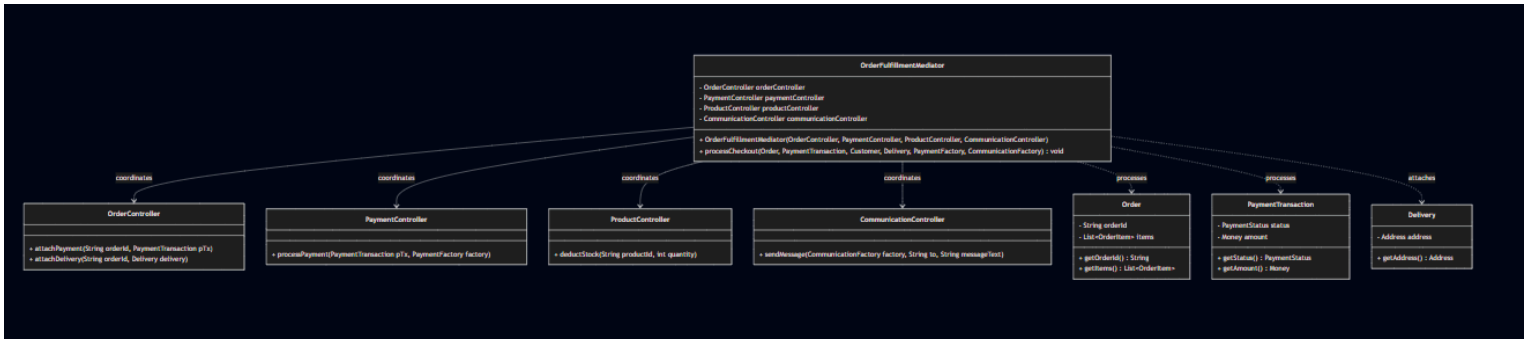
- **OrderFulfillmentMediator** → encapsulates and strictly executes the multi-step "Checkout Hub" workflow:
    - **Step 1:** Calls productController.deductStock(...) to secure physical inventory for the purchased items.
    - **Step 2:** Calls paymentController.processPayment(...) to attempt the financial transaction and validate the response.
    - **Step 3:** Calls orderController.attachPayment(...) and orderController.attachDelivery(...) to formally link the logistic and billing data to the system's order record.
    - **Step 4:** Calls communicationController.sendMessage(...) to generate and send a confirmation receipt to the customer.

Changes in Controller Classes

- **Decoupling:** The individual domain controllers (Order, Payment, Product, Communication) remain completely agnostic of the checkout scenario. They execute their narrow tasks without any knowledge of one another.
- **Delegation:** The client (Main.java) no longer has to figure out the correct sequence or handle failures between steps; it simply sets up the data and delegates execution by calling mediator.processCheckout().
- **Standardized Interaction:** The mediator utilizes pre-existing, distinct methods on the controllers to manipulate the data safely from the "outside."

Result

- **Decoupling**: Every controller remains strictly focused on its own domain, ensuring single responsibility. None of the four controllers import or reference one another.
- **Centralization**: The entire cross-module checkout workflow is highly visible, readable, and isolated inside a single Mediator class.
- **Scalability**: New steps (e.g., adding a fraud-detection check before payment, calculating shipping taxes, or alerting a warehouse agent) can easily be injected into the mediator workflow without modifying any existing core domain code.



Mediator Pattern: Complaint Escalation Coordination

Problem

In the CRM system, when a complaint's SLA deadline is breached Complaint.isSlaBreached() returns `true`, three separate actions must occur across three completely independent modules:

1. Customer module: the complaint must be reassigned to a supervisor via ComplaintController
2. Alert module: an SLA breach alert must be created via SlaBreachAlertController
3. Communication module: the customer must be notified about the escalation via CommunicationController

These three controllers have no knowledge of each other. Without a structured solution, the only place to write this coordination logic would be directly in Main.java, which would:

- Create tight coupling between unrelated modules, as Main.java would need to import and directly call all three controllers in sequence
- Turn Main.java into a God class a class that knows too much about the internals of every module
- Violate the Open/Closed Principle any new module that needs to react to SLA breaches would require modifying Main.java

- Make the system fragile and hard to maintain, since a developer could easily forget one of the three steps or call them in the wrong order

---

Solution

We applied the **Mediator Pattern** by:

- A ComplaintEscalationHandler interface was introduced with a single method onSlaBreached(). A concrete implementation ComplaintEscalationHandlerImpl was created to encapsulate the entire coordination workflow.
- Structure:
- **ComplaintEscalationMediator**: interface defining the coordination contract
- **ComplaintEscalationMediatorImpl**: holds private references to ComplaintController, SlaBreachAlertController, and CommunicationController all injected through the constructor.

Mediators Implemented

When **onSlaBreached**() is triggered, the mediator executes three steps in sequence:

- **Step 1:**Calls complaint.assignTo(supervisor) to reassign the complaint, then calls complaintController.updateComplaint() to persist the change
- **Step 2:** Calls slaAlertController.processAlert() to create an `SlaBreachAlert` and save it to the alert repository
- **Step 3:** Calls communicationController.sendMessage() using the provided communication factory to send an escalation notification to the customer

---

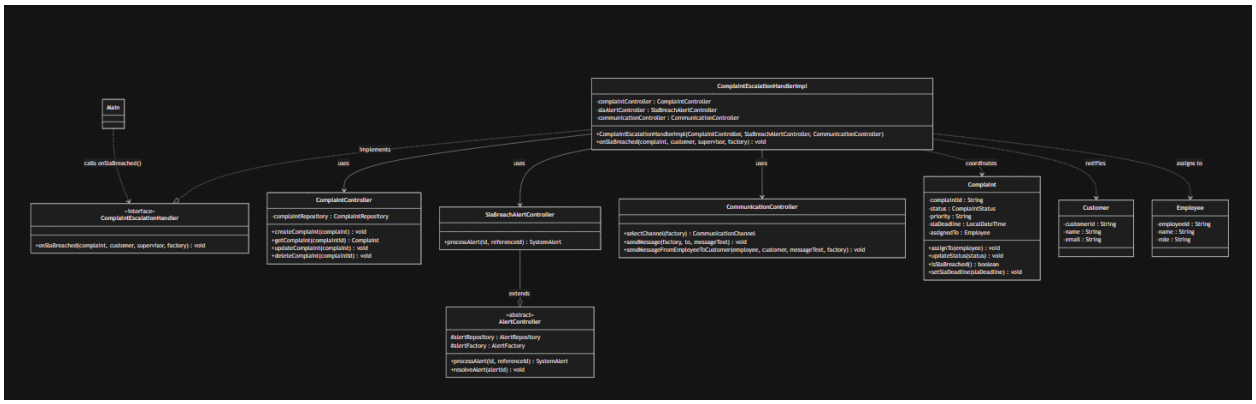
Changes in Controller Classes

The three controllers remain completely unchanged.None of them holds a reference to any of the others. The mediator is the only class that understands the full cross-module workflow.

Result

- **Decoupling**: ComplaintController, SlaBreachAlertController, and CommunicationController are fully independent from each other and interact only through the mediator
- **Single Responsibility**: each controller remains focused on its own domain; the mediator owns the orchestration logic exclusively

- **Extensibility**: if a future module needs to react to SLA breaches, only the mediator is updated; no existing controller is touched
- **Consistency**: the three-step escalation sequence is always executed in the correct order, guaranteed by the mediator, eliminating the risk of a caller missing a step



Observer + Singleton Pattern: Inventory Event Manager

Problem

Prior to adopting this pattern, tracking inventory changes across the system lacked a cohesive event-driven architecture.

This caused

- **Tight Coupling:** The Product entity (the catalogue) would have required hardcoded dependencies to various controllers just to inform them that stock had been reduced.
- **Inconsistent System State:** Whenever a product's stock was decreased, there was a high risk that the warehouse (InventoryRecord), alerting mechanisms, or the ERP system would not register the update unless developers remembered to call each controller manually.
- **Maintenance Difficulty:** Adding new features dependent on stock levels (e.g., automated analytics, delivery holds) would require directly modifying the Product.decreaseStock() method, breaking the Open-Closed Principle.

Solution

We applied the Observer Pattern through a centralized Singleton event bus InventoryEventManager by:

- Detaching the direct relationship between the product catalogue and backend business controllers.
- Providing a central hub where any subsystem can register to listen for inventory updates via the register() method**.**
- Updating Product to simply fire a global notification regarding its state change, effectively shifting from a direct procedure call model to an event-driven publish/subscribe model.

Event Workflow Product.decreaseStock(quantity): this.stockQuantity = Math.max(0, stockQuantity - quantity); InventoryEventManager.getInstance().notify(...); ,Broadcasts update to all subscribers

---

Event Interfaces Implemented

- **notify(productId, supplierId, newQuantity, unitPrice)** → Centralized in the Singleton event manager; wraps the parameters into a StockChangedEvent and pushes the event to all registered listeners.
- **onStockChanged(StockChangedEvent event)** → The interface method StockListener that every subscriber must implement to ingest and react to stock adjustments.

Subscribers Implemented

- **InventoryController**

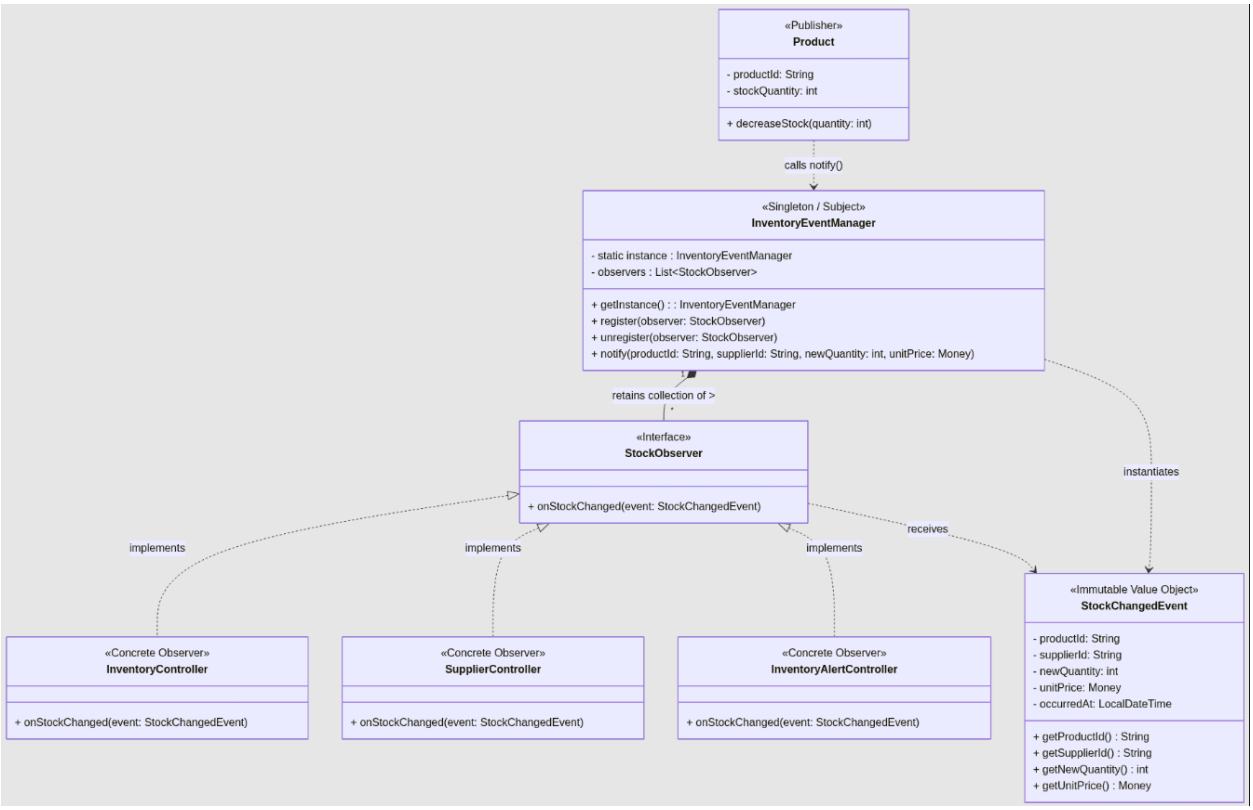
    - Subscribes to InventoryEventManager.
    - When notified, it compares the warehouse inventory (InventoryRecord) with the new product stock. If they are out of sync, it immediately auto-syncs the warehouse by applying the exact inventory difference.
- **InventoryAlertController**
    - Subscribes to InventoryEventManager.
    - When notified, it instantly processes and generates an automatic low-stock system alert (ALT-AUTO-...) specifically for the targeted product.
- **SupplierController**
    - Subscribes to InventoryEventManager.
    - When notified, it checks if the new quantity has dropped to 0 or below. If so, it automatically generates a PurchaseOrder (PO-AUTO-...) and directly delegates it to the external ERP system via the supplierDataService.

Changes in InventoryEventManager Class

- Configured as a Thread-Safe Singleton utilizing double-checked locking, making sure it serves as a single source of truth globally.
- Added an internal array list (List<StockListener> observers) to track registered subscribers.
- Created register() and unregister() methods.
- Refactored notify() to instantiate the StockChangedEvent object and push to the listener loops.

Result

- **Decoupling**: Product is solely responsible for catalogue updates; it pushes updates globally without needing any knowledge of alert systems or ERP setups.
- **Reactivity**: Multiple unconnected subsystems automatically and immediately react (Syncing warehouse stock, alerting the CRM, buying more stock via ERP) the instant a stock change occurs.
- **Efficiency**: Overcomes the need to manually execute identical warehouse syncs anywhere decreaseStock() was invoked in the client code.
- **Flexibility**: Adding a new service that relies on stock changes is as simple as attaching it as a new StockListener to the event manager.
- **Safety**: The global Singleton acts as a centralized traffic controller, cleanly isolating the event distribution logic.



Iterator Pattern: Order Items Traversal

Problem

Order.getItems() returns a raw Collections.unmodifiableList(items), exposing the entire internal list structure to every caller. Whenever a specific subset of items is needed

for example, items above a certain price, items for a specific product, or out-of-stock items every caller is forced to write their own filtering loop:

```
for (OrderItem item : order.getItems()) {

if (item.calculateSubtotal().getAmount() > 200) {}

}
```

This creates multiple problems:

- **Duplicated filtering logic**: the same threshold check is repeated across every caller
- **Exposed internals:** callers know and depend on the fact that items is a List, making it hard to change the internal structure

- **No consistency:**different callers may apply slightly different filter conditions for the same logical need
- **Violates Single Responsibility:** traversal and filtering logic does not belong in the calling code

---

Solution

A HVOrderItem class was created implementing java.util.Iterator<OrderItem>. A factory method highValueItemIterator(double minPrice) was added to Order to produce it

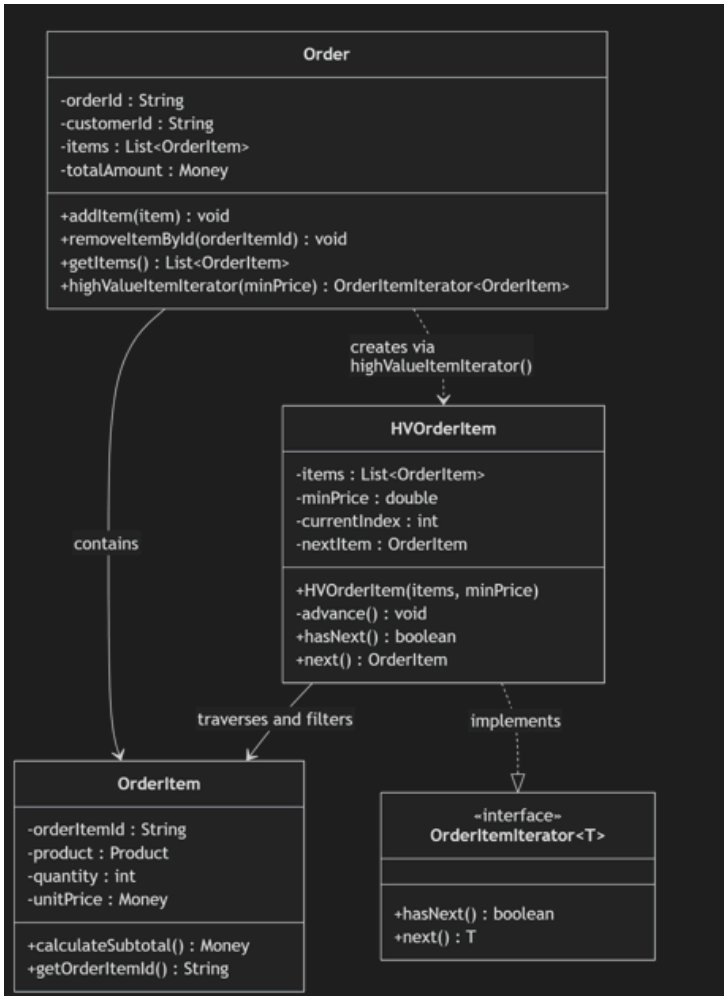
- HVOrderItem works internally receives the full item list and the minPrice threshold in the constructor
- Internally calls advance() which scans forward through the list to pre load the next qualifying item
- hasNext() simply checks if a qualifying item was found
- next() returns that item and immediately calls advance() again to prepare the next one
- Items whose calculateSubtotal().getAmount() does not exceed minPrice are silently skipped the caller never sees them

The caller has no knowledge of how filtering is done or that a List is involved at all

---

Result

- **Encapsulation**: the internal List<OrderItem> structure is fully hidden; callers only see the Iterator<OrderItem> interface
- **No duplicated logic**: the filtering threshold check lives exclusively inside HVOrderItem
- **Extensibility**: additional iterators (ProductFilterIterator, OutOfStockIterator) can be added by creating new classes and new factory methods on Order, without modifying any existing code
- **Standard Java interface**: uses java.util.Iterator<T>, so the iterator works seamlessly with any standard Java iteration consumer


Order
-orderId : String
-customerId : String
-items : List<OrderItem>
-totalAmount : Money
+addItem(item) : void
+removeItemById(orderItemId) : void
+getItems() : List<OrderItem>
+highValueItemIterator(minPrice) : OrderItemIterator<OrderItem>
creates via
highValueItemIterator()
HVOrderItem
-items : List<OrderItem>
-minPrice : double
-currentIndex : int
-nextItem : OrderItem
+HVOrderItem(items, minPrice)
-advance() : void
+hasNext() : boolean
+next() : OrderItem
contains
traverses and filters
implements
OrderItem
-orderItemId : String
-product : Product
-quantity : int
-unitPrice : Money
+calculateSubtotal() : Money
+getOrderItemId() : String
«interface»
OrderItemIterator<T>
+hasNext() : boolean
+next() : T